

Relatório Algoritmos Genéticos (Inteligência Artificial)

Gustavo Gurgel Medeiros - 539226

Jorge Eduardo Silva Sousa - 542051

Luiz Guilherme Moreira Leite - 537949

February 24, 2025

1 Implementação

O projeto foi desenvolvido utilizando a linguagem de programação Python. Foram utilizadas apenas bibliotecas nativas, sem dependências externas.

1.1 Inputs

Para executar os algoritmos genéticos, é necessário um arquivo **JSON** dentro da pasta **ga_inputs** com os seguintes atributos e com comentários explicando o que é cada um:

```
{
  "n": 3, // tamanho da entrada do problemas
  "selection_function": "SELECTION_WITH_ADDICTED_ROULETTE",
  "crossover_function": "CROSSOVER_WITH_TWO_POINTS",
  "mutation_function": "MUTATION_WITH_GLOBAL_SWAP",
  "elitism_function": "ELITISM_WITH_ONLY_BEST",
  "mutation_prob": 0.01, // probabilidade de mutacao
  "generation_limit": 4, // limite do numero geracoes
  "generation_size": 5, // tamanho da populacao na geracao
  "tournament_size": 3, // tamanho do torneio(em caso de uso de torneio)
  "crossover_majority_size": 3, // quantidade de individuos utilizados no
    crossover por maioria(em caso de uso de majority)
  "distance_matrix": [ // matrix de distancia(se passar sem, ira ser
    gerado aleatoriamente)
    [0, 10, 15],
    [10, 0, 12],
    [15, 12, 0]
  ],
  "flux_matrix": [ //matrix de fluxo(se passar sem, ira ser gerado
    aleatoriamente)
    [0, 3, 6],
    [3, 0, 1],
    [6, 1, 0]
  ]
}
```

Há alguns atributos que serão responsáveis pela seleção da função empregada em cada etapa do GA, a seguir sera listado cada um e suas respectivas opções de função:

- **selection_function:**
 - "SELECTION_WITH_ADDICTED_ROULETTE"

- "SELECTION_WITH_TOURNAMENT"
- `crossover_function`:
 - "CROSSOVER_WITH_TWO_POINTS"
 - "CROSSOVER_WITH_MAJORITY"
- `mutation_function`:
 - "MUTATION_WITH_GLOBAL_SWAP"
 - "MUTATION_WITH_LOCAL_SWAP"
- `elitism_function`:
 - "ELITISM_WITH_ONLY_BEST"
 - "ELITISM_WITH_ADDICTED_ROULETTE"

O que cada uma dessas funções fazem já é bem explícito pelo nome de cada uma, mas as particularidades serão explicadas de uma forma mais detalhada mais a frente.

1.2 Execução

Existem dois "solvers" na nossa implementação, o primeiro é o `ga_solver_qap.py`, ele roda na entrada apenas uma vez, mostrando informações importantes de cada geração, como: `best_fitness`, `worst_fitness`, `mean_fitness`, `mode_fitness`. Mais duas informações importantes que ele mostra é o `greedy_solution_distance` e o `greedy_solution_distance_percent`, que mostra a variação entre o nosso código e o algoritmo greedy, que na nossa opinião, serve para entender o quão vantajoso é usar o nosso algoritmo, e colocar em valores essa diferença. O `ga_solver_qap.py` tem alguns argumentos, um deles é obrigatório, e é o nome do arquivo de input que foi criado para ele rodar, e de opções tem o `-verbose`, que exibe detalhes da execução, o `-step`, que exibe o relatório de gerações passo a passo, o `-save` que salva o resultado num arquivo na pasta `ga_outputs` e por fim, o `-h` ou `-help`, que mostra todos os argumentos que descrevemos aqui, lá no terminal, para o caso de dúvidas na hora de executar.

O segundo "solver" é o `multi_ga_solver_qap_tester.py` que executa e compara dois arquivos de entrada JSON utilizando o GASolverQAP. Ele roda a entrada um número de vezes definido pelo usuário e calcula a média da diferença entre o algoritmo genético e o guloso, além do tempo médio de execução. Os argumentos obrigatórios são os nomes dos dois arquivos JSON de entrada e o número de execuções, que deve ser maior que 1. De opções, há o `-verbose`, que exibe detalhes da execução, e os clássicos `-h` ou `-help`, que mostram a lista de argumentos disponíveis diretamente no terminal. Ao final, o solver exibe a média de melhora do algoritmo genético em relação ao guloso para cada arquivo testado, junto com o tempo médio de execução.

2 Design dos Algoritmos

2.1 Variações para o Algoritmo Genético

2.1.1 Seleção Por Roleta Viciada

A roleta viciada (também conhecida como seleção por roleta ou roulette wheel selection) é um método probabilístico que seleciona indivíduos com base em sua aptidão (fitness). A ideia

central é que indivíduos com maior aptidão tenham uma chance proporcionalmente maior de serem escolhidos. No entanto, mesmo aqueles com menor aptidão ainda possuem uma pequena probabilidade de serem selecionados, garantindo diversidade na população.

O funcionamento desse método pode ser descrito da seguinte forma:

1. Soma-se a aptidão de todos os indivíduos na população.
2. Para cada indivíduo, calcula-se a proporção de sua aptidão em relação ao total.
3. Uma roleta virtual é girada, onde cada indivíduo ocupa um segmento proporcional à sua aptidão.
4. O indivíduo na posição onde a roleta parar é escolhido para reprodução.

A Figura 1 ilustra esse conceito por meio de um gráfico de pizza, onde cada fatia representa a probabilidade de seleção de um indivíduo com base em sua aptidão. **É importante resaltar que na figura quanto maior o fitness melhor o indivíduo. Porrem, no nosso caso essa proporção é inversa, ou seja, quanto menor o fitness, melhor.**

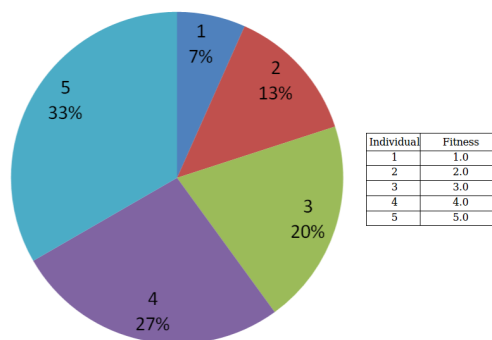


Figure 1: Exemplo de roleta viciada

2.1.2 Seleção Por Torneio

A seleção por torneio é um método baseado na competição entre um subconjunto aleatório da população. Ele não depende de cálculos probabilísticos complexos e é amplamente utilizado devido à sua simplicidade e eficiência.

O processo de seleção por torneio segue os seguintes passos:

1. Um número K de indivíduos é selecionado aleatoriamente da população.
2. Entre esses indivíduos, aquele com maior aptidão é escolhido como vencedor.
3. O vencedor avança para a próxima geração.

Esse método permite um maior controle sobre a pressão seletiva do algoritmo, pois quanto maior o tamanho do torneio (K), maior a probabilidade de que apenas os indivíduos mais aptos sejam escolhidos. Por outro lado, se K for pequeno, há uma maior diversidade na seleção.

A Figura 2 ilustra o funcionamento da seleção por torneio, mostrando um exemplo onde três indivíduos são selecionados aleatoriamente (K=3) e o mais apto entre eles é escolhido como vencedor.

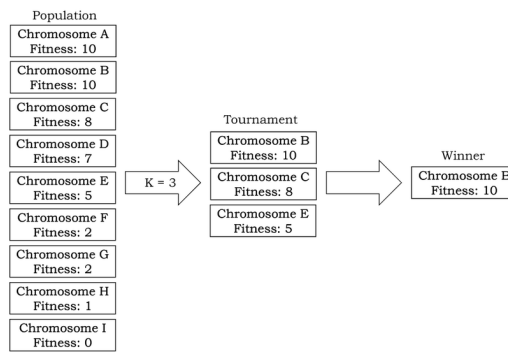


Figure 2: Exemplo de torneio

2.1.3 Crossover Por Dois Pontos

No crossover de dois pontos, cada pai é dividido em três partes através da definição de dois pontos de corte. O processo ocorre da seguinte maneira:

1. Dois pontos de corte são escolhidos aleatoriamente nos indivíduos pai.
2. A parte central de um pai é passada diretamente para o filho correspondente.
3. Os genes restantes do filho são preenchidos com os genes do outro pai, garantindo que não haja valores duplicados.

Como é visto na figura 3, esse método assegura que cada filho mantenha características de ambos os pais, preservando diversidade e evitando repetições indevidas de genes.

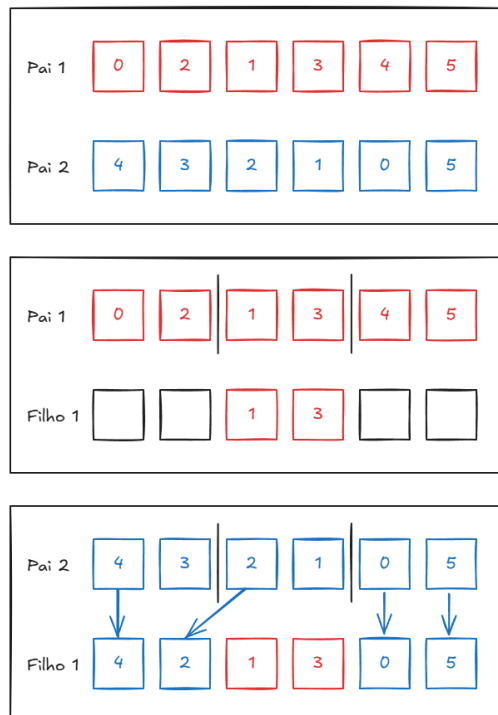


Figure 3: Crossover dois pontos

2.1.4 Crossover Por Maioria

O crossover por maioria adota um mecanismo diferente. Ele seleciona um conjunto de indivíduos e, para cada conjunto de genes na posição i , realiza um sorteio baseado no fitness de cada um. O funcionamento é descrito a seguir:

1. Um grupo de k indivíduos é escolhido para contribuir na formação dos filhos.
2. Um gene na posição i do conjunto de cromossomos pais é escolhido aleatoriamente com base no fitness de cada indivíduo.
3. O gene do pai com maior fitness tem maior probabilidade de ser escolhido para compor o cromossomo do filho.
4. Diferente do crossover de dois pontos, este método pode gerar genes duplicados, exigindo um mecanismo de correção para garantir cromossomos válidos.
5. Caso ocorra duplicação de valores, o algoritmo resolve substituindo valores repetidos por genes ausentes.

A figura 4 dá um exemplo desse método de crossover utilizando $k = 3$. Perceba que o indivíduo com menor fitness (melhor) contribuiu mais vezes do que os outros. Além disso, depois do cruzamento temos um conflito com dois valores 1's duplicados, que é resolvido logo em seguida.

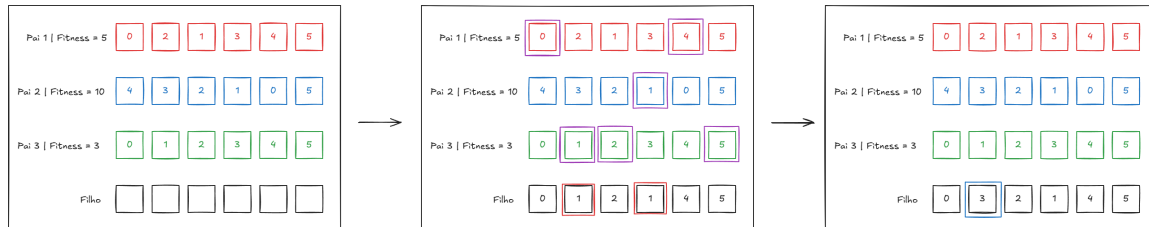


Figure 4: Exemplo crossover por maioria

2.1.5 Mutação de Probabilidade Global e Local

A **mutação global** ocorre com uma probabilidade definida para o **cromossomo como um todo**. Se a mutação for aplicada, dois genes dentro do cromossomo são sorteados aleatoriamente e trocados de posição. Essa abordagem permite **modificações mais amplas na solução**, promovendo maior exploração do espaço de busca e ajudando a escapar de mínimos locais.

Na **mutação local**, cada gene do cromossomo possui uma probabilidade específica de sofrer mutação. Caso um gene seja selecionado para mutação, ele será trocado com outro gene aleatório do cromossomo. Essa técnica permite uma **modificação mais gradual e localizada na solução**, o que pode favorecer a intensificação da busca em regiões promissoras do espaço de soluções.

A 5 mostra esses dois tipos de mutação em funcionamento

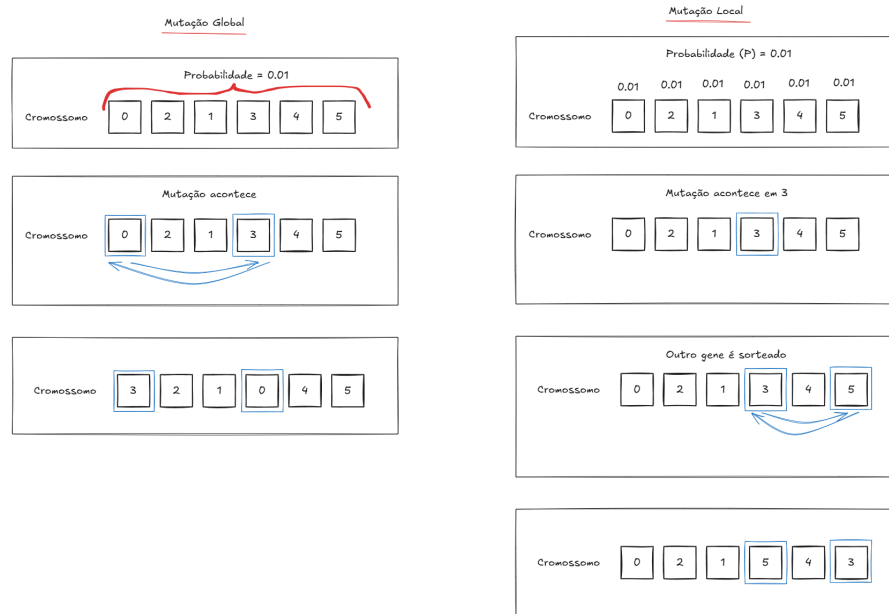


Figure 5: Exemplo mutação global e local

2.1.6 Elitismo Top-N (Apenas os melhores)

O Elitismo Top-N consiste em selecionar diretamente os N melhores indivíduos da população com base em sua aptidão e copiá-los para a próxima geração sem modificações. Essa abordagem assegura que as **melhores soluções não serão perdidas por conta de operadores genéticos como mutação e crossover**. No entanto, se N for grande demais, pode **levar à perda de diversidade genética**, dificultando a exploração do espaço de soluções.

Essa abordagem foi escolhida para **verificar se um método simples de retenção dos melhores indivíduos teria chances de encontrar boas soluções para o problema**. A ideia é avaliar se a mera preservação dos melhores indivíduos pode ser suficiente para alcançar resultados competitivos.

A figura 6 abaixo demonstra a seleção dos indivíduos para a geração 2

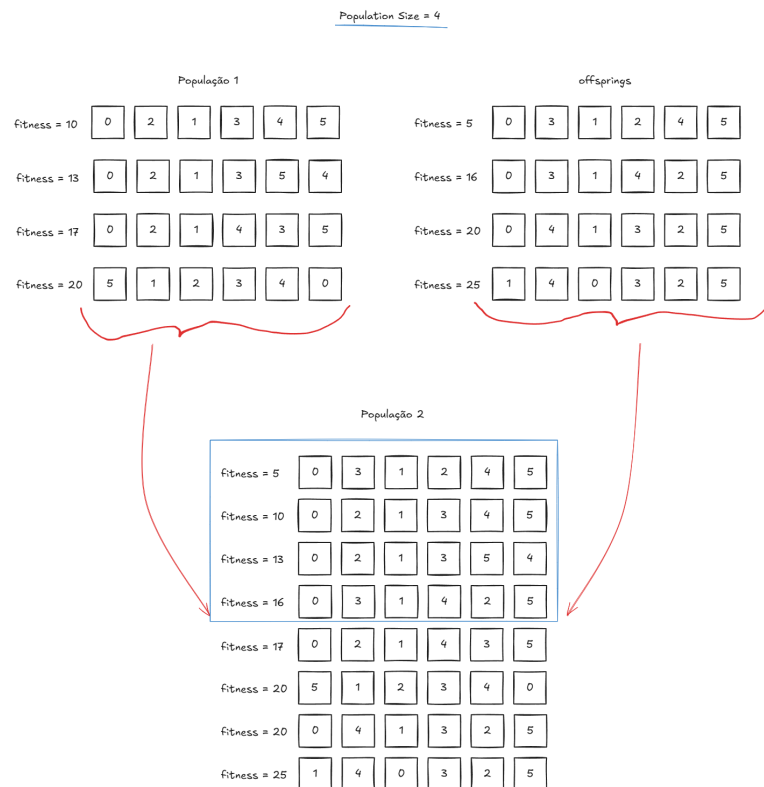


Figure 6: Exemplo elitismo Top-N

2.1.7 Elitismo Por Roleta Viciada

O Elitismo por Roleta Viciada funciona como uma versão modificada do método de roleta tradicional usado para seleção. Em vez de garantir diretamente a preservação dos melhores indivíduos, **este método aumenta a probabilidade de seleção dos indivíduos mais aptos na próxima geração, mantendo um equilíbrio entre preservação da elite e diversidade genética.** Dessa forma, os melhores indivíduos têm alta chance de serem mantidos, mas sem impedir que novas soluções promissoras possam surgir.

A figura 7 abaixo demonstra a seleção dos indivíduos para a geração 2

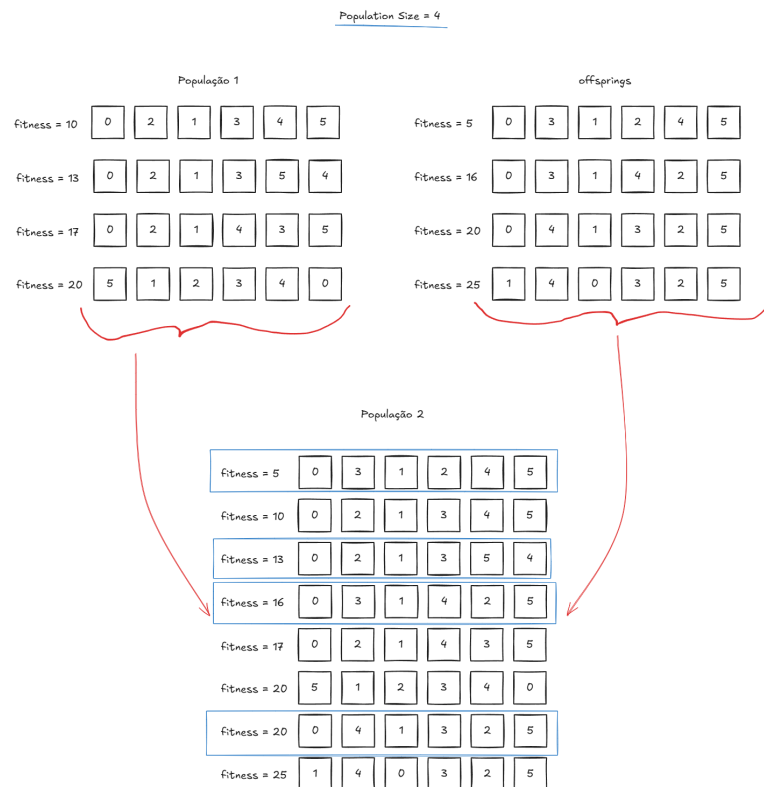


Figure 7: Exemplo elitismo com roleta viciada

2.2 Parte 0 - Escolha Dos Parâmetro Numéricos

Nesta seção, discutiremos a escolha dos parâmetros numéricos gerais que são aplicados a todos os algoritmos utilizados. Esses parâmetros incluem **generation_limit** (utilizado como critério de parada), **generation_size** e **mutation_prob**. Além disso, abordaremos também parâmetros numéricos opcionais, como **crossover_majority_size**, usado na função de crossover por maioria, e **tournament_size**, empregado na seleção por torneio.

Para garantir um padrão nos testes, utilizamos a seguinte configuração de entrada:

```

1 {
2   "n": 10,
3   "selection_function": "SELECTION_WITH_ADDICTED_ROULETTE",
4   "crossover_function": "CROSSOVER_WITH_MAJORITY",
5   "mutation_function": "MUTATION_WITH_GLOBAL_SWAP",
6   "elitism_function": "ELITISM_WITH_ADDICTED_ROULETTE",
7   "mutation_prob": 0.01,
8   "generation_limit": 10,
9   "generation_size": 10,
10  "crossover_majority_size": 3,
11  "tournament_size": 3
12 }
```

Figure 8: Inputs padrões utilizados nos testes

O parâmetro $n = 10$ indica que estamos resolvendo um problema Quadratic Assignment Problem (QAP) de tamanho 10. A escolha desse tamanho se deve ao fato de que,

com uma entrada relativamente pequena, conseguimos realizar uma bateria de testes muito maior, permitindo inferências mais robustas sobre os resultados. Além disso, **generation_limit = 10** e **generation_size = 10** foram definidos como valores base para as experimentações.

Cada experimentação foi realizada utilizando a configuração padrão mencionada acima, variando um único parâmetro de cada vez. **Para cada variação, foram executados 100 testes aleatórios, medindo a média do acréscimo de fitness em relação a uma solução gulosa** baseada na heurística de menor fluxo para o maior caminho. Além disso, também registramos a média do tempo de execução para cada configuração.

2.2.1 Escolha do parâmetro generation_limit (critério de para)

generation_limit	Média Ganho Guloso (%)	Média Tempo Execução (s)
3	1.4415	0.0041
5	2.2275	0.0076
10	2.8945	0.0168
15	2.9895	0.0275
20	2.3460	0.0342
25	2.6215	0.0404
30	3.0795	0.0502
35	3.2030	0.0548
40	1.9355	0.0747
45	3.4920	0.0857
50	3.0185	0.0857
55	3.4775	0.1140
60	3.8010	0.1193
65	3.6675	0.1274
70	3.4365	0.1326
75	2.7575	0.1209
80	3.0500	0.1365
85	3.2670	0.1551
90	2.9610	0.1417
95	3.0975	0.1469

Table 1: Resultados para diferentes valores de generation_limit

Analisando os dados para $n = 10$, observamos que os melhores resultados ocorrem na faixa de generation_limit entre 45 e 60. Decedimos pegar generation_limit=50 por ficar no meio desses dois valores.

2.2.2 Escolha do parâmetro `generation_size`

<code>generation_size</code>	Média do Ganho Guloso	Média do Tempo de Execução(s)
3	-3.2695	0.00505
5	-2.4210	0.00781
10	1.0230	0.01672
15	3.6295	0.02402
20	3.6270	0.03211
25	4.2530	0.03663
30	4.9850	0.04434
35	5.0555	0.05202
40	4.7890	0.06871
45	4.7535	0.07052
50	5.4115	0.07657
55	5.0760	0.09884
60	6.4175	0.10137
65	5.6020	0.12753
70	5.6120	0.13080
75	6.4400	0.14176
80	5.9385	0.14312
85	6.0140	0.12980
90	5.8120	0.14780
95	5.9460	0.16086

Table 2: Variação do parâmetro `generation_size` para $n = 10$

A partir dos resultados apresentados, observa-se que `generation_size = 60` obteve um dos melhores ganhos médios sobre a solução gulosa (6.4175) sem aumentar significativamente o tempo de execução em comparação com valores superiores de `generation_size`. Valores acima de 60, como 75 e 95, mantiveram ganhos médios similares, porém, com um custo computacional crescente. Assim, a escolha de **`generation_size = 60`** se justifica por proporcionar um bom equilíbrio entre ganho e tempo de execução.

2.2.3 Escolha do parâmetro `mutation_prob`

<code>mutation_prob</code>	Média Ganho Guloso	Média Tempo Execução (s)
0.001	0.528	0.0164
0.005	0.951	0.0168
0.01	1.023	0.0167
0.05	2.057	0.0172
0.1	2.894	0.0178
0.2	3.214	0.0185
0.3	3.056	0.0187
0.4	2.897	0.0191
0.5	2.741	0.0198
0.6	2.203	0.0204
0.7	1.689	0.0212
0.8	1.312	0.0219
0.9	0.784	0.0226

Table 3: Resultados médios para diferentes valores de `mutation_prob`

Os resultados indicam que um valor muito baixo de **`mutation_prob`** (por exemplo, 0.001) não introduz variações suficientes na população, levando a um menor ganho médio em relação à solução gulosa. Por outro lado, valores excessivamente altos (acima de 0.5) podem perturbar a estrutura das soluções boas, reduzindo o ganho de fitness e aumentando o tempo de execução sem benefícios substanciais.

O melhor desempenho foi observado com **`mutation_prob`** em torno de **0.05**, onde obtivemos um bom compromisso entre exploração e estabilidade da busca. Esse valor proporcionou um ganho médio de **2.057** sobre a solução gulosa, mantendo o tempo de execução em um patamar aceitável. Lembrando que não podemos escolher valores muito altos porque se não o algoritmo ficaria muito randômico

2.2.4 Escolha do Parâmetro *tournament_size*

<i>tournament_size</i>	Média Ganho Guloso	Média Tempo Execução (s)
3	-0.2560	0.02049
5	-0.7390	0.02402
10	-0.6215	0.02404
15	-1.6955	0.02167
20	-1.8825	0.01942
25	-0.8050	0.02072
30	-1.2915	0.02060
35	-1.9590	0.02182
40	-2.0140	0.02307
45	-1.8180	0.02237
50	-1.1955	0.02387
55	-1.8700	0.02149
60	-2.5605	0.02199
65	-1.3485	0.02252
70	-1.8000	0.02087
75	-1.5360	0.02029
80	-1.2035	0.02172
85	-1.8950	0.02206
90	-1.9005	0.02295
95	-1.3310	0.02462

Table 4: Resultados para diferentes valores de *tournament_size*

Com base nos dados apresentados, buscamos um equilíbrio entre o ganho em relação à solução gulosa e o tempo de execução.

Observamos que o valor 3 conseguiu se aproximar melhor da solução gulosa. Lembrando que para esse teste foram utilizados *generation_size*= 10 e *generation_limit*= 10 que são valores muito baixos, o que explica a solução ser pior que a gulosa.

Portanto, considerando a relação entre tempo de execução e ganho, o valor ideal de *tournament_size* escolhido foi 3, pois apresenta um dos melhores ganhos, mantendo um tempo muito baixo.

2.2.5 Análise do Parâmetro `crossover_majority_size`

<code>crossover_majority_size</code>	Média Ganho Guloso(%)	Média Tempo Execução (s)
3	1.859	0.0171
5	1.9025	0.0197
10	1.754	0.0198
15	2.401	0.0235
20	1.425	0.0209
25	1.280	0.0204
30	1.7405	0.0213
35	1.157	0.0235
40	2.2145	0.0236
45	0.7165	0.0246
50	0.472	0.0280
55	1.1915	0.0326
60	1.832	0.0341
65	1.066	0.0304
70	1.435	0.0330
75	1.832	0.0326
80	1.0165	0.0362
85	1.4155	0.0371
90	2.5005	0.0360
95	1.9245	0.0390

Table 5: Resultados para diferentes valores de `crossover_majority_size`

A partir dos resultados, observamos que o maior ganho em relação à solução gulosa foi obtido com `crossover_majority_size` = 90, atingindo um valor médio de 2.5005. No entanto, o tempo de execução médio para essa configuração foi de 0.0360 segundos, um dos maiores registrados.

Por outro lado, a configuração com `crossover_majority_size` = 15 apresentou um segundo melhor ganho médio de 2.401, com um tempo de execução de apenas 0.0235 segundos, um valor consideravelmente menor. Assim, optamos por recomendar `crossover_majority_size` = 15, pois apresenta um excelente equilíbrio entre ganho e tempo de execução, maximizando a eficiência do algoritmo sem comprometer significativamente o desempenho computacional.

2.3 Conclusão dos valores escolhidos na Parte 0

Concluindo, para um entrada de tamanho $n = 10$ os melhores valores em questão de **tempo de execução e qualidade da resposta** são:

- `generation_limit`: 50
- `generation_size`: 60
- `mutation_prob`: 0.05
- `tournament_size`: 3
- `crossover_majority_size`: 15

3 Experimentação

Neste conjunto de experimentos, buscamos analisar o impacto de diferentes parâmetros nos algoritmos genéticos aplicados ao Problema Quadrático de Alocação (QAP). A estrutura dos experimentos segue uma abordagem progressiva, **onde os resultados de cada etapa influenciam as escolhas da próxima**. Assim, conseguimos refinar continuamente os parâmetros utilizados para obter um melhor desempenho.

3.1 Parte 1

Execução	Roleta Viciada	Fitness	Torneio	Fitness
1	[8, 2, 0, 6, 5, 1, 3, 7, 4, 9]	4147	[5, 0, 4, 8, 3, 2, 6, 9, 7, 1]	4086
2	[3, 2, 0, 1, 4, 6, 8, 7, 5, 9]	3101	[6, 9, 0, 4, 1, 5, 3, 7, 8, 2]	2968
3	[8, 1, 6, 3, 0, 4, 2, 7, 5, 9]	2831	[0, 4, 2, 6, 3, 1, 7, 5, 8, 9]	2769
4	[3, 6, 1, 8, 4, 9, 5, 2, 0, 7]	3714	[5, 0, 4, 8, 7, 3, 1, 2, 9, 6]	3429
5	[8, 4, 3, 1, 0, 2, 7, 5, 9, 6]	3982	[1, 8, 9, 6, 2, 0, 3, 4, 7, 5]	3795
6	[5, 0, 2, 1, 3, 8, 4, 7, 6, 9]	3744	[7, 6, 1, 0, 2, 3, 5, 4, 8, 9]	3629
7	[3, 0, 4, 5, 2, 1, 7, 9, 6, 8]	3429	[9, 5, 2, 0, 8, 7, 4, 3, 1, 6]	3171
8	[2, 6, 1, 0, 4, 7, 3, 5, 8, 9]	3260	[2, 8, 5, 0, 4, 7, 3, 1, 6, 9]	3103
9	[8, 4, 9, 0, 6, 2, 5, 1, 3, 7]	3795	[8, 4, 9, 3, 2, 6, 5, 1, 0, 7]	3746
10	[6, 2, 0, 3, 4, 5, 7, 9, 1, 8]	4073	[5, 0, 2, 3, 1, 4, 7, 9, 6, 8]	3983
11	[3, 2, 1, 7, 6, 0, 8, 9, 5, 4]	3760	[5, 2, 8, 3, 6, 4, 1, 9, 0, 7]	3692
12	[4, 1, 6, 0, 3, 5, 2, 9, 8, 7]	3793	[5, 6, 1, 8, 2, 4, 0, 3, 7, 9]	3599
13	[7, 2, 0, 3, 4, 1, 6, 9, 8, 5]	3440	[3, 7, 9, 0, 1, 8, 2, 5, 4, 6]	3209
14	[9, 2, 0, 3, 1, 7, 4, 5, 8, 6]	3218	[6, 0, 2, 1, 3, 4, 9, 8, 5, 7]	3091
15	[6, 7, 4, 2, 0, 3, 1, 5, 9, 8]	2407	[6, 1, 3, 2, 0, 8, 4, 9, 5, 7]	2390
16	[6, 4, 0, 3, 8, 7, 1, 2, 5, 9]	3081	[2, 3, 6, 0, 5, 7, 1, 4, 8, 9]	3054
17	[1, 9, 8, 3, 2, 0, 5, 7, 4, 6]	3464	[6, 1, 8, 0, 4, 5, 3, 2, 9, 7]	3398
18	[2, 1, 7, 8, 3, 0, 4, 5, 6, 9]	2816	[2, 1, 4, 3, 8, 6, 9, 7, 5, 0]	2820
19	[0, 2, 1, 3, 4, 6, 5, 9, 7, 8]	3952	[6, 9, 1, 8, 4, 0, 7, 3, 2, 5]	3738
20	[4, 2, 1, 5, 0, 8, 9, 3, 6, 7]	3593	[4, 6, 2, 0, 3, 8, 1, 5, 9, 7]	3459

Table 6: Comparação entre Seleção por Roleta Viciada e Seleção por Torneio

A análise dos resultados mostra que a **seleção por torneio** teve um desempenho superior à **seleção por roleta viciada**. Os resultados médios indicam que a seleção por torneio conseguiu uma melhora média de 6.61% sobre o algoritmo guloso, enquanto a roleta viciada teve uma melhora de apenas 3.23%. Além disso, o tempo médio de execução da seleção por torneio (1.4299s) foi significativamente menor do que o da roleta viciada (2.2560s).

Dessa forma, podemos concluir que a **seleção por torneio** apresenta vantagens tanto em eficiência quanto em tempo de execução, sendo a melhor escolha para este problema.

3.2 Parte 2

Execução	Maioria	Fitness	Dois Pontos	Fitness
1	[4, 1, 0, 8, 7, 5, 3, 6, 2, 9]	3990	[3, 8, 4, 1, 7, 5, 0, 9, 2, 6]	4094
2	[3, 0, 8, 1, 7, 6, 4, 5, 2, 9]	2969	[1, 3, 5, 7, 2, 6, 9, 4, 0, 8]	2966
3	[1, 0, 3, 2, 8, 7, 5, 4, 6, 9]	3161	[4, 9, 3, 5, 7, 1, 8, 6, 0, 2]	3182
4	[7, 8, 5, 4, 0, 9, 1, 3, 6, 2]	2931	[6, 7, 1, 9, 8, 0, 3, 5, 2, 4]	2956
5	[7, 9, 1, 8, 0, 2, 5, 6, 3, 4]	3200	[2, 1, 6, 8, 5, 0, 4, 3, 9, 7]	3172
6	[2, 3, 4, 1, 7, 0, 5, 9, 6, 8]	3652	[8, 1, 4, 3, 7, 5, 0, 2, 9, 6]	3640
7	[3, 5, 9, 7, 0, 4, 2, 1, 6, 8]	3275	[8, 1, 3, 9, 0, 2, 6, 5, 4, 7]	3308
8	[6, 2, 3, 8, 9, 7, 0, 5, 1, 4]	2780	[3, 6, 4, 5, 8, 9, 7, 2, 1, 0]	2700
9	[7, 1, 0, 9, 3, 4, 6, 5, 2, 8]	3327	[6, 9, 0, 7, 3, 8, 4, 5, 2, 1]	3295
10	[9, 8, 0, 6, 1, 4, 3, 2, 5, 7]	3221	[0, 9, 2, 5, 1, 7, 6, 4, 8, 3]	3387
11	[1, 4, 0, 7, 2, 3, 6, 8, 5, 9]	4476	[2, 6, 1, 0, 7, 4, 8, 3, 5, 9]	4574
12	[8, 0, 4, 1, 2, 9, 5, 3, 6, 7]	3367	[1, 2, 8, 0, 5, 6, 4, 7, 3, 9]	3283
13	[3, 6, 5, 1, 0, 2, 7, 4, 9, 8]	3359	[7, 3, 6, 9, 0, 2, 8, 5, 1, 4]	3341
14	[6, 4, 8, 1, 0, 3, 7, 2, 9, 5]	2605	[4, 7, 8, 6, 1, 3, 9, 2, 0, 5]	2669
15	[6, 4, 0, 1, 7, 8, 3, 5, 9, 2]	3265	[6, 8, 9, 5, 7, 2, 4, 0, 1, 3]	3316
16	[5, 3, 1, 7, 0, 2, 4, 6, 9, 8]	3648	[0, 3, 4, 6, 7, 2, 9, 8, 1, 5]	3678
17	[3, 0, 1, 7, 6, 2, 9, 4, 5, 8]	3121	[3, 4, 0, 8, 1, 7, 2, 5, 6, 9]	3134
18	[7, 2, 1, 3, 5, 4, 9, 0, 6, 8]	3725	[4, 6, 3, 2, 7, 5, 9, 1, 8, 0]	3667
19	[6, 9, 0, 7, 8, 3, 2, 1, 5, 4]	3358	[2, 8, 0, 7, 5, 3, 6, 9, 1, 4]	3472
20	[2, 1, 7, 3, 6, 9, 0, 4, 5, 8]	4300	[3, 4, 8, 0, 6, 2, 5, 1, 9, 7]	4407

Table 7: Comparação entre as funções de Crossover: Majority versus Two Points

A análise dos resultados revela que o uso da função **CROSSOVER_WITH_MAJORITY** proporcionou, em média, uma melhora de 6.56% sobre o algoritmo guloso, com um tempo médio de execução de 1.4095 segundos. Em contrapartida, a função **CROSSOVER_WITH_TWO_POINTS** apresentou uma melhora média de 5.93%, porém com um tempo médio de execução menor, de 1.0695 segundos.

Dessa forma, podemos concluir que, enquanto a abordagem **Majority** tende a alcançar soluções com um ganho percentual superior, a abordagem **Two Points** oferece maior eficiência computacional. A escolha entre elas dependerá do balanço desejado entre qualidade da solução e tempo de processamento para o problema em questão.

3.3 Parte 3

Execução	Only-Best	Fitness	Addicted-Roulette	Fitness
1	[6, 9, 0, 5, 3, 1, 2, 4, 7, 8]	2795	[7, 4, 3, 2, 0, 1, 5, 9, 6, 8]	2750
2	[5, 4, 3, 0, 8, 2, 7, 1, 6, 9]	4086	[5, 0, 1, 9, 3, 6, 4, 2, 7, 8]	4110
3	[7, 0, 6, 3, 2, 4, 9, 1, 5, 8]	2624	[7, 0, 1, 3, 6, 4, 9, 2, 5, 8]	2633
4	[9, 1, 0, 3, 5, 2, 8, 4, 7, 6]	3241	[2, 1, 9, 3, 5, 0, 8, 4, 7, 6]	3271
5	[1, 3, 8, 0, 5, 4, 2, 7, 6, 9]	3342	[4, 3, 0, 6, 8, 2, 9, 5, 1, 7]	3348
6	[1, 3, 9, 5, 7, 0, 4, 2, 6, 8]	2957	[0, 3, 6, 2, 5, 1, 4, 8, 9, 7]	2970
7	[0, 4, 1, 8, 5, 3, 9, 6, 2, 7]	3141	[9, 4, 2, 3, 5, 0, 8, 1, 6, 7]	3186
8	[4, 6, 0, 2, 7, 5, 1, 8, 3, 9]	3069	[8, 1, 9, 2, 3, 5, 4, 6, 7, 0]	3102
9	[2, 8, 4, 9, 0, 6, 5, 3, 7, 1]	2903	[5, 4, 9, 7, 6, 1, 2, 8, 0, 3]	2932
10	[4, 9, 2, 0, 3, 1, 7, 5, 8, 6]	3342	[8, 4, 0, 7, 1, 3, 9, 5, 6, 2]	3429
11	[9, 2, 1, 3, 7, 0, 5, 6, 4, 8]	4007	[8, 1, 9, 0, 3, 4, 6, 7, 2, 5]	4084
12	[4, 2, 7, 0, 5, 1, 3, 6, 8, 9]	3058	[6, 4, 2, 7, 5, 1, 8, 0, 3, 9]	3074
13	[9, 0, 4, 7, 6, 8, 3, 1, 5, 2]	4237	[3, 7, 4, 0, 1, 9, 2, 5, 6, 8]	4347
14	[2, 8, 7, 3, 9, 0, 5, 4, 1, 6]	3438	[9, 8, 2, 5, 3, 0, 6, 4, 7, 1]	3535
15	[9, 5, 8, 0, 7, 4, 1, 2, 3, 6]	2920	[4, 2, 3, 5, 1, 9, 7, 0, 6, 8]	2986
16	[0, 6, 1, 8, 2, 3, 4, 9, 5, 7]	3408	[5, 7, 1, 3, 9, 4, 2, 8, 0, 6]	3481
17	[1, 0, 9, 3, 5, 4, 7, 8, 6, 2]	4146	[0, 4, 6, 5, 3, 7, 2, 8, 9, 1]	4149
18	[1, 9, 3, 0, 6, 7, 4, 2, 8, 5]	3973	[3, 2, 0, 7, 1, 8, 6, 4, 5, 9]	4182
19	[0, 3, 1, 4, 5, 7, 6, 8, 2, 9]	2653	[5, 7, 1, 8, 0, 3, 6, 4, 2, 9]	2736
20	[9, 6, 4, 5, 1, 2, 3, 0, 7, 8]	3769	[4, 1, 6, 9, 8, 5, 7, 0, 3, 2]	3852

Table 8: Comparação entre as Variações Only-Best e Addicted-Roulette

Este estudo comparou duas variações de elitismo de algoritmos genéticos: **ONLY_BEST**, que preserva apenas o melhor indivíduo, e **ADDICTED_ROULETTE**, que utiliza uma roleta viciada para seleção. Os resultados de 20 execuções independentes mostram que o **ONLY_BEST** superou o **ADDICTED_ROULETTE** na maioria das vezes (19 contra 1), encontrando soluções com melhor fitness (8.62% de melhoria em relação a 7.25% do **ADDICTED_ROULETTE** em relação ao algoritmo guloso).

Embora o tempo de execução do **ADDICTED_ROULETTE** tenha sido ligeiramente menor, a diferença não é significativa. A eficácia superior do **ONLY_BEST** pode ser atribuída à preservação da melhor solução, acelerando a convergência, apesar do risco de estagnação. A roleta viciada, por outro lado, mantém a diversidade genética, mas pode levar a uma convergência mais lenta e soluções menos otimizadas.

Dessa forma, podemos concluir que o **ONLY_BEST** apresenta vantagens tanto em eficiência quanto em tempo de execução, sendo a melhor escolha para este problema. Em conclusão, para o problema analisado, a variação **ONLY_BEST** demonstrou um desempenho superior, indicando que a preservação do melhor indivíduo é mais eficaz na busca por soluções de alta qualidade.

3.4 Parte 4

Execução	GLOBAL	Fitness	LOCAL	Fitness
1	[3, 4, 9, 1, 2, 0, 7, 6, 8, 5]	3552	[6, 7, 3, 2, 4, 5, 1, 0, 9, 8]	3489
2	[4, 7, 3, 0, 2, 6, 1, 5, 9, 8]	2805	[3, 7, 6, 0, 2, 1, 8, 4, 9, 5]	2782
3	[9, 6, 5, 1, 8, 7, 3, 0, 4, 2]	3228	[7, 0, 3, 4, 1, 5, 6, 9, 8, 2]	3176
4	[9, 8, 7, 6, 1, 0, 5, 2, 4, 3]	3021	[9, 8, 7, 6, 1, 0, 5, 2, 4, 3]	3021
5	[0, 8, 7, 5, 9, 3, 6, 4, 2, 1]	2933	[9, 0, 7, 5, 1, 4, 6, 2, 3, 8]	2962
6	[3, 6, 1, 8, 5, 2, 4, 7, 0, 9]	3930	[3, 6, 1, 7, 8, 2, 5, 4, 0, 9]	3928
7	[9, 2, 3, 6, 1, 4, 5, 8, 7, 0]	3927	[6, 0, 8, 9, 1, 2, 3, 4, 7, 5]	3953
8	[3, 8, 4, 5, 9, 6, 7, 2, 0, 1]	2647	[9, 7, 6, 3, 1, 5, 8, 0, 4, 2]	2633
9	[7, 2, 0, 4, 8, 1, 5, 9, 6, 3]	3038	[7, 2, 0, 4, 8, 1, 5, 9, 6, 3]	3038
10	[7, 6, 1, 2, 0, 3, 5, 9, 4, 8]	2908	[7, 1, 3, 9, 0, 8, 5, 2, 4, 6]	2903
11	[3, 7, 0, 1, 5, 8, 4, 6, 2, 9]	2442	[3, 7, 0, 1, 5, 8, 4, 6, 2, 9]	2442
12	[7, 9, 8, 0, 3, 4, 1, 2, 6, 5]	2618	[8, 5, 2, 6, 3, 4, 1, 0, 9, 7]	2634
13	[1, 2, 5, 0, 9, 4, 6, 3, 7, 8]	2599	[2, 1, 5, 0, 9, 4, 6, 7, 3, 8]	2587
14	[7, 5, 9, 2, 4, 3, 8, 0, 1, 6]	2552	[7, 5, 9, 2, 4, 3, 8, 0, 1, 6]	2552
15	[2, 1, 7, 0, 3, 5, 4, 8, 6, 9]	4011	[4, 3, 8, 7, 2, 5, 1, 9, 0, 6]	4000
16	[5, 0, 9, 1, 3, 7, 6, 8, 4, 2]	3404	[6, 0, 3, 8, 4, 9, 2, 1, 7, 5]	3423
17	[4, 9, 0, 2, 8, 3, 1, 7, 6, 5]	2257	[8, 1, 0, 2, 4, 3, 9, 7, 5, 6]	2257
18	[4, 7, 3, 1, 2, 6, 8, 0, 9, 5]	2871	[1, 0, 7, 3, 4, 6, 5, 8, 9, 2]	2845
19	[1, 6, 0, 5, 2, 8, 3, 9, 4, 7]	3825	[1, 6, 0, 7, 5, 4, 3, 9, 8, 2]	3811
20	[0, 2, 1, 7, 6, 4, 5, 3, 8, 9]	3216	[0, 4, 5, 8, 2, 1, 9, 7, 6, 3]	3189

Table 9: Comparação entre as funções de Mutação: Global Swap vs. Local Swap

A análise dos resultados revela que o uso da função **MUTATION_WITH_GLOBAL_SWAP** proporcionou, em média, uma melhora de 8.61% sobre o algoritmo guloso, com um tempo médio de execução de 1.9763 segundos. Já a função **MUTATION_WITH_LOCAL_SWAP** apresentou uma melhora ligeiramente superior, de 8.83%, porém com um tempo de execução maior, de 2.4709 segundos.

Dessa forma, a escolha entre essas abordagens depende da prioridade entre tempo de execução e qualidade da solução, sendo que o método **Global Swap** é mais rápido, enquanto o **Local Swap** obtém soluções um pouco melhores.

3.5 Parte 5 - Estimativa do Tempo Máximo Viável

A tabela abaixo apresenta os tempos de execução e os valores de fitness encontrados para diferentes tamanhos de entrada n nos quatro experimentos realizados.

n	Parte 1	Parte 1 Fit	Parte 2	Parte 2 Fit	Parte 3	Parte 3 Fit	Parte 4	Parte 4 Fit	Greedy Fit
10	0.5415 s	3005	0.5986 s	3073	0.6836 s	3011	0.7317 s	2991	3180
15	2.1195 s	8505	2.1866 s	8503	2.8336 s	8831	2.6214 s	8252	8840
20	5.9059 s	16404	5.7061 s	16667	7.6065 s	16929	7.3079 s	16114	17671
25	13.2031 s	24697	13.1723 s	24550	17.1822 s	25130	17.1913 s	23902	24782
30	26.9565 s	32009	26.9495 s	31828	35.1564 s	33255	35.1359 s	31653	33289
35	49.4599 s	53667	49.5809 s	52863	64.5618 s	53979	64.5606 s	52766	54499
40	83.9246 s	64088	83.8220 s	65230	109.5271 s	66312	109.3343 s	64695	65637
45	135.9566 s	76472	135.3473 s	76989	176.4130 s	78645	179.4772 s	77123	78906
50	206.1258 s	90393	207.0836 s	89649	271.6046 s	92100	269.9092 s	90594	92266
55	301.7980 s	120252	304.8117 s	121405	459.8688 s	122119	409.7465 s	120862	120517
60	442.6448 s	153135	442.5026 s	154563	635.6078 s	155749	633.6828 s	154728	154077

Table 10: Resultados dos testes para diferentes tamanhos de entrada n

A partir dos dados coletados considerando apenas a execução da parte 4 (que obteve melhor rendimento), realizamos um ajuste de curva para estimar o tempo de execução em função do tamanho da entrada n . A equação obtida foi:

$$f(n) = 0.43784n^2 - 19.4389n + 194.606 \quad (1)$$

A Figura 9 apresenta o gráfico dessa função sobreposto aos pontos coletados da parte 4, demonstrando que a modelagem se aproxima bem dos valores medidos.

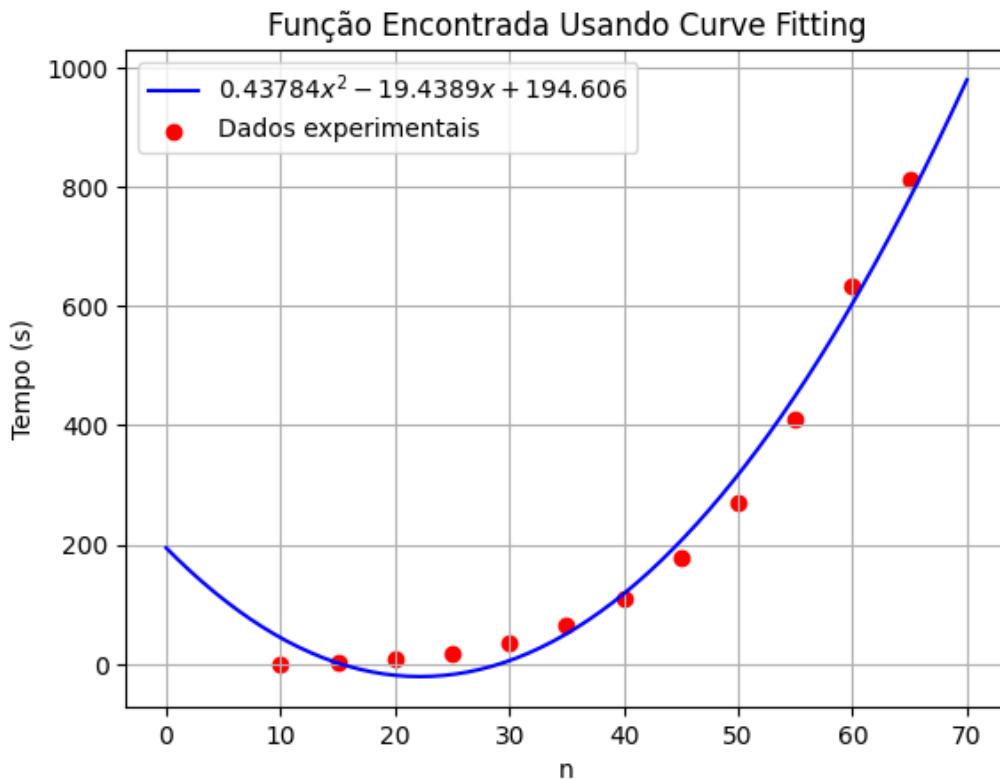


Figure 9: Ajuste de curva do tempo de execução em função do tamanho da entrada n

Para estimar um limite prático para n , consideramos o tempo máximo de execução de um mês (aproximadamente 2.629.744 segundos). Resolvendo a equação:

$$0.43784n^2 - 19.4389n + 194.606 = 2.629.744 \quad (2)$$

Isolando n , obtemos a equação quadrática:

$$0.43784n^2 - 19.4389n - 2.629.549.394 = 0 \quad (3)$$

Utilizando a fórmula de Bhaskara:

$$n = \frac{-(-19.4389) \pm \sqrt{(-19.4389)^2 - 4(0.43784)(-2.629.549.394)}}{2(0.43784)} \quad (4)$$

Calculando o valor da raiz positiva:

$$n \approx 2473 \quad (5)$$

Isso significa que, baseado na extrapolação dos tempos de execução, a configuração da parte 4 para entradas com valor maior ou igual a 2473 demoraria cerca de um mês para serem resolvidas, o que é um tempo não hábil.